Capítulo 8: Personalización de Git

Actividad 2 - Basado en Pro Git (Chacon & Straub, 2014) - Juan Manuel Gomez Perez - 43972

Introducción

Git se puede adaptar al estilo de trabajo de cada persona o equipo mediante opciones de configuración, atributos y ganchos (hooks) que automatizan acciones.

8.1 Configuración de Git

```
git config --global core.editor "code --wait"
git config --global commit.template ~/.gitmessage.txt
git config --global core.pager "less -F"
git config --global core.excludesfile ~/.gitignore_global
git config --global help.autocorrect 1
git config --global color.ui auto
git config --global user.signingkey <ID-de-clave>
```

Opciones como el editor, la plantilla de mensajes, el paginador y los colores personalizan la experiencia diaria. La configuración del servidor también permite fijar políticas, por ejemplo receive.denyNonFastForwards.

8.2 Atributos de Git

El archivo .gitattributes define comportamientos por ruta o patrón de archivo:

- Marcar archivos como binarios para que no se muestren diferencias de texto.
- Definir cómo comparar formatos especiales (por ejemplo, documentos de Word) con filtros externos.
- Normalizar finales de línea entre sistemas operativos.
- Aplicar filtros de "limpieza" y "manchado" (clean/smudge) o exportar con exclusiones usando export-ignore.

```
*.docx diff=word
*.png binary
*.sh text eol=lf
```

8.3 Puntos de enganche (hooks)

Los hooks son scripts que Git ejecuta automáticamente ante ciertos eventos. Se guardan en .git/hooks y deben ser ejecutables.

Hooks del lado del cliente

- pre-commit: se ejecuta antes de confirmar; útil para pruebas y revisiones de estilo.
- prepare-commit-msg y commit-msg: permiten preparar o validar el mensaje.
- post-commit: notificaciones tras confirmar.
- pre-rebase, post-checkout, post-merge: acciones en torno a esas operaciones.

Hooks del lado del servidor

- pre-receive y update: pueden rechazar un push que no cumpla las políticas.
- post-receive: se ejecuta tras recibir los cambios; sirve para notificar o desplegar.

8.4 Ejemplo de política aplicada con Git

Con un hook commit-msg o update se puede exigir, por ejemplo, que los mensajes sigan un formato determinado, que solo ciertos usuarios modifiquen ciertas rutas o que no se acepten cambios que no sean de avance rápido.

Fuente: Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress.
https://git-scm.com/book/es/v2